

漏洞审计报告

目标: Flask 用户管理系统 (app.py)

框架: Flask + SQLite3

审计日期: 2026-07-08

一、本次重点修复：SQL 注入漏洞

当日教学内容知识点：SQL 注入原理、利用方式与参数化查询防御

1.1 漏洞原理

SQL 注入 (SQL Injection, CWE-89) 是指攻击者通过将恶意的 SQL 语句片段注入到应用程序的输入参数中，欺骗后端数据库执行非预期查询的攻击方式。

核心成因：开发者使用字符串拼接 (Python f-string 或 `+` 运算符) 将用户输入直接嵌入 SQL 语句，未做任何转义或参数化处理。数据库引擎无法区分「代码」与「数据」，攻击者输入的内容被当作 SQL 代码执行。

危害：攻击者可绕过认证、窃取数据、篡改记录、删除表甚至执行系统命令 (视数据库权限而定)。

本系统存在两处 SQL 注入点，分别位于搜索功能和注册功能。

1.2 漏洞位置

漏洞点 ①: `/search` 搜索接口

文件: 源码-未修复/app.py

行号: 第 639 行

```
sql = f"SELECT * FROM users WHERE username LIKE '%{keyword}%' OR email LIKE '%{keyword}%'"
cursor.execute(sql)
```

`keyword` 来自 `request.args.get("keyword", "")` (第 628 行)，未经任何处理直接拼入 SQL 语句。

漏洞点 ②: `/register` 注册接口

文件: 源码-未修复/app.py

行号: 第 608 行

```
sql = f"INSERT INTO users (username, password, email, phone) VALUES ('{username}', '{password}', '{email}', '{phone}')"
cursor.execute(sql)
```

四个字段 `username`、`password`、`email`、`phone` 全部来自用户 POST 表单数据，直接拼入 INSERT 语句。

1.3 攻击方式

攻击演示 ①：搜索接口 UNION 注入 — 窃取全表数据

搜索接口无需登录即可访问（另一缺陷），攻击者直接在搜索框输入 payload：

Payload:

```
' UNION SELECT id, username, password, password, phone FROM users --
```

构造的完整 SQL:

```
SELECT * FROM users WHERE username LIKE '%' UNION SELECT id, username, password, password, phone FROM users -- '%' OR email LIKE '%...%'
```

攻击效果: 数据库中所有用户的明文密码被直接返回到搜索结果中:

ID	用户名	密码(明文)	邮箱	手机
1	admin	admin123	admin@example.com	13800138000
2	alice	alice2025	alice@example.com	13900139001

攻击演示 ②：搜索接口布尔盲注 — 逐字符猜解数据

当应用不直接返回数据但存在行为差异时可用:

```
' AND (SELECT substr(password,1,1) FROM users WHERE username='admin')='a' --
```

通过遍历字符和观察返回结果是否为空，可逐字符推断出 admin 的密码。

攻击演示 ③：注册接口恶意 INSERT — 插入管理员账户

攻击者在注册表单的 `username` 字段中注入:

```
attacker', 'hash_attacker', 'attacker@evil.com', '13800000000'); DELETE FROM users WHERE username='admin'; --
```

构造的完整 SQL:

```
INSERT INTO users (username, password, email, phone) VALUES ('attacker', 'hash_attacker', 'attacker@evil.com', '13800000000'); DELETE FROM users WHERE username='admin'; -- ', '...', '...', '...')
```

攻击效果: 插入恶意用户的同时删除 admin 账户 (SQLite3 默认允许多语句执行)。

1.4 修复方案

核心方案：参数化查询 (Parameterized Query / Prepared Statement)

将所有 SQL 语句中的用户输入替换为 `?` 占位符，通过 `cursor.execute(sql, params)` 的第二个参数传入实际值。数据库驱动负责将参数值安全地绑定，确保用户输入永远被当作「数据」而非「SQL 代码」处理。

具体措施：

1. 将所有 `f-string` 拼接方式改为 `?` 占位符 + 参数元组绑定
2. 搜索 SQL 改为显式列名（排除密码字段，防止敏感信息泄露）
3. 抽取数据库操作到独立辅助函数，集中管理 SQL 语句
4. 注册入口增加 CSRF 校验、输入长度限制、业务规则校验等多层防御

1.5 修复代码

修复 ①：搜索接口 (`flask-user-mgmt-fixed/app.py` 第 267-282 行)

```
def db_search_users(keyword):
    """根据 username 和 email 进行模糊搜索，使用参数化查询"""
    conn = get_db()
    try:
        # 过滤 LIKE 通配符，防止批量导出或逐字符枚举
        safe_keyword = keyword.replace("%", "").replace("_", "")
        if len(safe_keyword) < 1:
            return []
        pattern = f"%{safe_keyword}%"
        rows = conn.execute(
            "SELECT uid, username, email, phone FROM users WHERE username LIKE ?
            OR email LIKE ?",
            (pattern, pattern)
        ).fetchall()
        return [dict(row) for row in rows]
    finally:
        conn.close()
```

修复前：

```
sql = f"SELECT * FROM users WHERE username LIKE '%{keyword}%' OR email LIKE '%{keyword}%'"
cursor.execute(sql)
```

修复要点：

- 使用 `?` 占位符替代 `f-string` 拼接
- `SELECT` 显式指定列名 (`uid, username, email, phone`)，不再返回密码字段
- 过滤 `%` 和 `_` 通配符，防止攻击者利用 `LIKE` 进行批量匹配或逐字符枚举

修复 ②：注册接口 (flask-user-mgmt-fixed/app.py 第 193-209 行)

```
def db_insert_user(uid, username, password_hash, email, phone):
    """插入新用户, 返回 (success, error_message)"""
    conn = get_db()
    try:
        conn.execute(
            "INSERT INTO users (uid, username, password, email, phone) VALUES"
            "(?, ?, ?, ?, ?)",
            (uid, username, password_hash, email, phone)
        )
        conn.commit()
        return True, ""
    except sqlite3.IntegrityError:
        return False, "用户名已存在"
    except Exception as e:
        logger.error(f"插入用户失败: {e}")
        return False, "注册失败, 请稍后再试"
    finally:
        conn.close()
```

修复前:

```
sql = f"INSERT INTO users (username, password, email, phone) VALUES"
    ('{username}', '{password}', '{email}', '{phone}')"
cursor.execute(sql)
```

修复要点:

- 使用 `?` 占位符替代 f-string 拼接
- 调用方 (register 路由) 已对输入做长度校验、密码强度校验、XSS 过滤等多层验证
- 注册增加 CSRF Token 校验 (第 649 行)
- 密码在入库前已通过 `generate_password_hash()` 哈希处理

1.6 效果验证

验证 ①：搜索接口注入测试

测试输入 (搜索框输入)：

```
' UNION SELECT uid, username, password, uid, phone FROM users --
```

预期结果: 搜索返回空结果或无匹配, SQL 注入 payload 被当作普通字符串进行 LIKE 匹配, 不会触发 UNION 注入。

验证方法: 访问 `/search?keyword=' UNION SELECT ...`, 检查返回结果中不应包含数据库其他用户的密码信息。日志中记录的 SQL 为参数化形式。

验证 ②：注册接口注入测试

测试输入（注册表单 username 字段）：

```
test'); DROP TABLE users; --
```

预期结果：

- 若用户名长度超限（>50），被前端校验拦截
- 若通过长度校验，该字符串作为普通用户名尝试插入，参数化查询将其视为数据值，数据库存储为普通用户名

验证方法：注册该用户名后检查数据库 users 表是否仍存在（未被删除），且新用户记录中 username 字段即为该字符串面值。

二、其他同步发现并修复的问题

非本次教学重点，但在代码审计过程中同步发现并修复的安全问题。

2.1 明文密码存储（CWE-312）— 高危

漏洞位置：源码-未修复/app.py 第 111-112 行

问题描述：数据库初始化时将 admin 和 alice 用户的密码以明文形式硬编码写入 SQLite：

```
cursor.execute("INSERT OR IGNORE INTO users (username, password, ...) VALUES ('admin', 'admin123', ...)")
cursor.execute("INSERT OR IGNORE INTO users (username, password, ...) VALUES ('alice', 'alice2025', ...)")
```

一旦 SQL 注入成功或被获取数据库文件，攻击者直接拿到明文凭据。

修复：初始密码改为从环境变量 `INIT_PWD_ADMIN` / `INIT_PWD_ALICE` 读取，经 `generate_password_hash()` 哈希后存入数据库（`flask-user-mgmt-fixed/app.py` 第 82-102 行）。源码中不再出现任何明文密码。

2.2 搜索接口使用 `SELECT *` 泄露密码字段（CWE-200）— 中危

漏洞位置：源码-未修复/app.py 第 639 行

问题描述：`SELECT * FROM users` 返回全部列包括 `password`，Row 转 dict 后传入模板上下文。虽然模板未渲染密码字段，但数据已流入不可信通道。

修复：改为显式列名 `SELECT uid, username, email, phone FROM users`，从查询层杜绝密码字段泄露（`flask-user-mgmt-fixed/app.py` 第 277 行）。

2.3 注册接口缺失 CSRF 保护 (CWE-352) — 中危

漏洞位置: 源码-未修复/app.py 第 597-621 行

问题描述: login 和 logout 端点均有 CSRF 校验, 但 /register 端点完全未调用 `validate_csrf()` 和 `generate_csrf_token()`, 攻击者可构造恶意页面迫使用户在不知情下注册账户或利用注入点执行攻击。

修复: register 路由的 POST 处理中增加 `validate_csrf()` 校验 (flask-user-mgmt-fixed/app.py 第 649-650 行) 。

2.4 搜索接口无认证控制 (CWE-862) — 低危

漏洞位置: 源码-未修复/app.py 第 626-650 行

问题描述: /search 无需登录即可访问, 未登录用户也能执行搜索和注入攻击, 放大了 SQL 注入的利用面。

修复: 搜索路由开头检查 `session.get("uid")`, 未登录直接重定向到登录页 (flask-user-mgmt-fixed/app.py 第 726-728 行) 。

2.5 Debug 模式无条件绑定 (CWE-489) — 中危

漏洞位置: 源码-未修复/app.py 第 14 行

问题描述: ProxyFix 中间件无条件启用, 若本应用部署在非代理环境中, `x_for=1` 可能导致客户端 IP 伪造。

修复: 改为通过环境变量 `BEHIND_PROXY` 控制是否启用 ProxyFix (flask-user-mgmt-fixed/app.py 第 14-15 行) 。

2.6 输入缺少 XSS 过滤 (CWE-79) — 中危

漏洞位置: 源码-未修复/app.py 注册和个人信息更新接口

问题描述: 用户输入的 username、email、phone 等字段未经 XSS 过滤直接存入数据库, 后续渲染到页面时可能导致存储型 XSS。

修复: 增加 `sanitize_input()` 函数, 拒绝包含 HTML 标签的输入, 并对邮箱、手机字段使用白名单正则校验 (flask-user-mgmt-fixed/app.py 第 518-530 行) 。

2.7 Logout 通过 GET 请求触发 (CWE-352) — 中危

漏洞位置: 源码-未修复/app.py 第 458 行

问题描述: 原代码注释已指出 GET /logout 可被 `` 强制登出, 已改为 POST + CSRF。但确保修复一致性, 与 register 的 CSRF 修复配合。

修复: Logout 路由仅接受 POST, 并校验 CSRF Token (flask-user-mgmt-fixed/app.py 第 763-770 行) 。

2.8 登录无时序侧信道防护（CWE-208） — 低危

漏洞位置: 源码-未修复/app.py 第 419-424 行

问题描述: 用户存在与否的代码分支执行时间不同，攻击者可通过测量响应时间进行用户名枚举。

修复: 加入假哈希常量 DUMMY_HASH，用户不存在时也执行 check_password_hash()，保证两分支执行路径时序一致（flask-user-mgmt-fixed/app.py 第 610-616 行）。

2.9 登录限流仅 IP 维度 — 可被代理池绕过（CWE-307） — 中危

漏洞位置: 源码-未修复/app.py 暴力破解防御逻辑

问题描述: 仅按 IP 限制登录尝试次数，攻击者使用代理池轮换 IP 即可绕过。

修复: 增加用户名维度的限流（PER_USERNAME_LIMIT），任一维度触发即锁定（flask-user-mgmt-fixed/app.py 第 598-608 行）。

2.10 缺少安全响应头 — 低危

漏洞位置: 源码-未修复/app.py 全局响应处理

问题描述: 未设置 X-Content-Type-Options、X-Frame-Options、X-XSS-Protection、HSTS 等安全响应头。

修复: 在 after_request 钩子中统一添加安全响应头（flask-user-mgmt-fixed/app.py 第 337-347 行）。

漏洞总览

#	漏洞	级别	原始位置	修复文件位置	CWE
1	SQL 注入 (搜索) ●	严重	旧 app.py:639	db_search_users():277	CWE-89
2	SQL 注入 (注册) ●	严重	旧 app.py:608	db_insert_user():198	CWE-89
3	明文密码存储	高	旧 app.py:111-112	init_db():91-101	CWE-312
4	SELECT * 泄露密码	中	旧 app.py:639	db_search_users():277	CWE-200
5	注册缺失 CSRF	中	旧 app.py:597	register():649	CWE-352
6	搜索无认证控制	中	旧 app.py:626	search():726-728	CWE-862
7	Debug模式 ProxyFix	中	旧 app.py:14	app.py:14-15	CWE-489

#	漏洞无 XSS 过滤	级别	原始位置个人信息接口	修复代码位置 sanitize_input():518-530	CWE-79
9	Logout GET 方式	中	旧 app.py:458	logout():763-770	CWE-352
10	时序侧信道	低	旧 app.py:419-424	login():610-616	CWE-208
11	登录限流可绕过	低	旧暴力破解防御逻辑	login():598-608	CWE-307
12	缺失安全响应头	低	旧全局响应处理	remove_server_header():337	CWE-693

● = 本次重点 (SQL 注入, 当日教学内容)